

Практическая работа №7

Управление компоновкой

Введение в виджеты: <https://metanit.com/dart/flutter/2.1.php>

Чтобы пойти дальше, надо прочитать эту небольшую статью.

Виджеты — это основные строительные блоки приложений Flutter. Все в приложении Flutter — это виджет, начиная с текста на экране и заканчивая кнопками и контейнерами, которые их содержат. Они используются для описания того, как должен выглядеть пользовательский интерфейс и как он должен себя вести.

По своей сути виджеты — это классы, которые определяют часть пользовательского интерфейса. У них есть метод сборки, который возвращает дерево виджетов, описывающее пользовательский интерфейс. Дерево виджетов состоит из других виджетов, которые могут быть расположены в иерархии для создания сложного пользовательского интерфейса.

Вот пример виджета, который отображает простое текстовое сообщение:

```
class MyText extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Text('Hello, World!');  
  }  
}
```

Давайте подробно рассмотрим из чего он состоит:

1. Предоставленный код определяет новый класс виджета `MyText`, который наследует виджет с классом `StatelessWidget`. Это означает, что виджет является неизменяемым и не имеет никакого изменяемого состояния.
2. `Widget` — это базовый строительный блок пользовательского интерфейса в Flutter. Он представляет собой часть пользовательского интерфейса и отвечает за описание того, как он должен отображаться и как он должен себя вести.
3. Метод `build()` является обязательным методом для создания виджета во Flutter. Он отвечает за возврат дерева виджетов, которое описывает пользовательский интерфейс виджета. Метод `build()` вызывается фреймворком, когда виджет необходимо обновить или перестроить заново.
4. `BuildContext` — это контекстный объект, который передается методу `build()` и используется для определения местоположения виджета в дереве виджетов.

Он также используется для предоставления доступа к различным ресурсам, таким как темы, медиа-запросы и локализации.

5. `context` — это ссылка на текущий `BuildContext`. Он предоставляет информацию о положении виджета в дереве виджетов и может быть использован для получения информации о родительских, дочерних или родственных виджетах виджета. контекст часто используется для извлечения различных ресурсов или для доступа к навигатору для навигации между различными экранами в приложении.
6. В методе `build()` класса `myText` возвращается новый текстовый виджет `Text`. Текстовый виджет используется для отображения текста «Hello, world!» на экране.

Основные виджеты во Flutter

Ниже приведены некоторые из наиболее часто используемых виджетов во Flutter вместе с их краткими описаниями:

- **Text:** Этот виджет используется для отображения текстовой строки на экране.
- **Container:** Виджет-контейнер используется для группировки других виджетов вместе и применения к ним таких свойств, как отступы, поля, граница и цвет фона.
- **Image:** Этот виджет используется для отображения изображений в приложении Flutter. Он поддерживает различные форматы, такие как PNG, JPEG и GIF.
- **Row** и **Column:** Эти виджеты используются для расположения других виджетов в горизонтальном (строка) или вертикальном (столбец) направлении. Они позволяют вам создавать макеты с несколькими виджетами, которые могут быть выровнены и разнесены в соответствии с вашими потребностями.
- **ListView:** Этот виджет используется для создания прокручиваемых списков виджетов. Это очень полезный виджет для отображения больших объемов данных в ограниченном пространстве.
- **TextFormField:** Этот виджет используется для сбора входных данных от пользователей. Это похоже на традиционное поле ввода текста, но предоставляет больше возможностей настройки, таких как проверка, форматирование ввода и сообщения об ошибках.
- **FloatingActionButton:** Этот виджет используется для создания кнопки, которая обычно размещается в правом нижнем углу экрана. Обычно он используется для запуска таких действий, как добавление элемента, отправка сообщения или совершение звонка.

- **InkWell:** Этот виджет используется для создания интерактивной области, которая может реагировать на действия пользователя, такие как касание, двойное касание или длительное нажатие. Он часто используется для создания кнопок или ссылок, которые имеют пользовательские визуальные эффекты, такие как анимация пульсаций.

Это лишь некоторые из наиболее часто используемых виджетов во Flutter. В фреймворке Flutter доступно еще много виджетов, каждый со своим собственным набором свойств и вариантов использования.

Немного про MaterialApp

MaterialApp — это виджет в Flutter, который определяет базовую структуру приложения Material Design. Он предоставляет ряд общих функций приложения, таких как навигатор для управления переходами экрана, тема для определения стилей для всего приложения и система локализации для управления переводами приложений.

Чтобы использовать MaterialApp в вашем проекте Flutter, вам необходимо импортировать пакет **material.dart** в свой код. Вы можете сделать это, добавив следующую инструкцию `import` в начало файла `main.dart`:

При создании нового приложения Flutter первым виджетом, который вы обычно создаете, является виджет MaterialApp. Вы можете задать различные свойства виджета MaterialApp, такие как название приложения, тема и начальный экран для отображения. Виджет MaterialApp также предоставляет доступ к различным функциям уровня приложения, таким как навигатор для управления переходами экрана и `LocalizationDelegate` для управления переводами приложений.

Вот пример базового виджета MaterialApp:

```
MaterialApp(  
  title: 'My App',  
  theme: ThemeData(  
    primarySwatch: Colors.blue,  
  ),  
  home: MyHomePage(),  
);
```

В этом примере мы устанавливаем для названия приложения значение «My App», а для темы используем образец основного цвета — синий. Мы также устанавливаем для

свойства `home` пользовательский виджет `MyHomePage()`, который будет первым экраном, отображаемым в приложении.

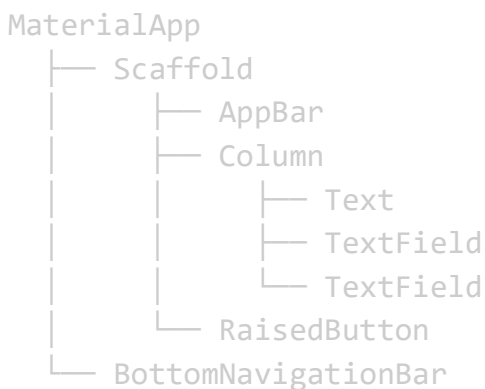
`MaterialApp` во Flutter принимает несколько пар ключ-значение в качестве именованных аргументов.

Дерево виджетов во Flutter

В Flutter дерево виджетов — это иерархическая структура виджетов, которая описывает пользовательский интерфейс приложения. Корнем дерева виджетов обычно является виджет `MaterialApp` или `CupertinoApp`, который определяет основной внешний вид приложения, включая тему приложения и структуру навигации.

Пример дерева виджетов

Вот пример дерева виджетов, который представляет из себя страницу авторизации:



Давайте рассмотрим структуру подробнее:

1. В этом примере `Material App` служит корнем дерева виджетов. Он определяет основной внешний вид приложения, включая тему приложения и структуру навигации.
2. Внутри `MaterialApp` у нас есть виджет `Scaffold`, который обеспечивает базовую структуру макета для экрана входа в систему. `Scaffold` содержит виджет панели приложений `AppBar`, который отображает заголовок приложения и другие элементы управления навигацией.
3. Внутри `Scaffold` есть виджет `Column`, который размещает свои дочерние виджеты вертикально. Столбец содержит текстовый виджет, который

отображает заголовок экрана входа в систему, а также два виджета текстовых полей для пользователя, чтобы ввести свой адрес электронной почты и пароль.

4. В нижней части каркаса у нас есть виджет кнопки `RaisedButton`, который позволяет пользователю отправлять свои учетные данные для входа.
5. Приложение `MaterialApp` также включает виджет `BottomNavigationBar`, который предоставляет элементы управления навигацией для других экранов приложения.

В целом, это дерево виджетов представляет собой простой экран входа в систему, созданный с использованием комбинации повторно используемых виджетов.

Комбинируя и вкладывая виджеты таким образом, разработчики могут создавать сложные и отзывчивые пользовательские интерфейсы во Flutter.

Как выглядит дерево виджетов в коде

Вот как будет выглядеть код на Flutter для предоставленного нами дерева виджетов:

```
MaterialApp(  
  home: Scaffold(  
    appBar: AppBar(  
      title: Text('My App'),  
    ),  
    body: Column(  
      children: [  
        Text('Hello'),  
        TextField(),  
        TextField(),  
        RaisedButton(  
          onPressed: () {},  
          child: Text('Submit'),  
        ),  
      ],  
    ),  
    bottomNavigationBar: BottomNavigationBar(  
      items: [  
        BottomNavigationBarItem(  
          icon: Icon(Icons.home),  
          title: Text('Home'),  
        ),  
        BottomNavigationBarItem(  
          icon: Icon(Icons.search),  
          title: Text('Search'),  
        ),  
      ],  
    ),  
  ),  
)
```

```

    ),
    BottomNavigationBarItem(
      icon: Icon(Icons.settings),
      title: Text('Settings'),
    ),
  ],
),
),
);

```

Давайте подробнее рассмотрим основные компоненты в коде:

- `home` — это свойство Material App, которое определяет виджет, который будет отображаться в качестве домашней страницы приложения. В нашем примере он установлен на виджет Scaffold.
- `body` — это свойство виджета Scaffold, которое определяет основное содержимое приложения. Обычно он устанавливается на виджет, который может содержать другие виджеты, такие как столбец, ListView или контейнер.
- `bottomNavigationBar` — это свойство виджета Scaffold, которое определяет панель навигации в нижней части приложения.
- `children` — это свойство многих виджетов макета, таких как Column, Row и ListView, которое определяет список дочерних виджетов, которые будут отображаться в этом макете. В нашем примере виджет столбца имеет три дочерних виджета: два виджета текстового поля и виджет с приподнятой кнопкой.

В целом, дерево виджетов является важной частью архитектуры Flutter, основанной на виджетах, и понимание того, как создавать деревья виджетов и управлять ими, необходимо для разработки высококачественных приложений Flutter.

Типы виджетов во Flutter

Во Flutter существует два основных типа виджетов: **StatfullWidget** (с сохранением состояния) и **StatelessWidget** (без сохранения состояния).

- **StatfullWidget:** Виджет без состояния — это виджет, который не меняется с течением времени. Виджеты без состояния обычно используются для простых элементов пользовательского интерфейса, таких как кнопки, текст и значки. Эти виджеты неизменяемы, что означает, что их свойства не могут быть

изменены после их создания. Примеры виджетов без состояния в Flutter включают текст, изображение, иконку и RaisedButton.

- **StatelessWidget:** Виджет с сохранением состояния — это виджет, который может меняться с течением времени. Эти виджеты используются для более сложных элементов пользовательского интерфейса, требующих динамического поведения или взаимодействия, таких как формы или анимация. Виджеты с отслеживанием состояния поддерживают свое собственное состояние, которое может быть изменено на протяжении всего жизненного цикла виджета. Примеры виджетов с отслеживанием состояния в Flutter включают ListView, TextField и Checkbox.

Разберем основные виджеты:

Text: <https://metanit.com/dart/flutter/3.1.php>

Container: <https://metanit.com/dart/flutter/2.6.php>

Column: <https://metanit.com/dart/flutter/2.7.php>

Row: <https://metanit.com/dart/flutter/2.8.php>

Image: <https://metanit.com/dart/flutter/3.6.php>

Задание:

Мини-проект: Карточка товара

Задача: Создайте карточку товара, используя только изученные виджеты.

Требования:

- Заголовок товара (Text).
- Картинка товара
- Описание (Text в Container с отступами).
- Цена и рейтинг (два Text в Row).
- Всё это должно быть внутри Column и обернуто в Container с рамкой (decoration: BoxDecoration(border: Border.all())).

